A

Technical Seminar Report

On

A MALWARE DETECTION APPROACH USING AUTOENCODER IN DEEP LEARNING

Submitted

To

Jawaharlal Nehru Technological University, Hyderabad

in partial fulfillment of the requirements for the award of Degree of

Bachelor of Technology

in

Computer Science & Engineering (AI&ML)

BY

DASI SUSMITHA (216Y5A6601)

Department of Computer Science & Engineering

SUMATHI REDDY
INSTITUTE OF TECHNOLOGY FOR WOMEN
Learning at its best
Ananthasagar, Hasanparthy, Warangal -506371, Telangana. Website: www.sritw.org
Affiliated to JNTUH - Approved by AICTE

2023-2024

SUMATHI REDDY
INSTITUTE OF TECHNOLOGY FOR WOMEN
Learning at its best
Ananthasagar, Hasanparthy, Warangal -506371, Telangana. Website: www.sritw.org
Affiliated to JNTUH - Approved by AICTE

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is certify that the Seminar report entitled "**A MALWARE DETECTION APPROACH USING AUTOENCODER IN DEEP LEARNING**" is submitted by **D.SUSMITHA (216Y5A6601)**, in the partial fulfillment of requirement for the award of degree of Bachelor of Technology in Computer Science and Engineering(AI&ML) during academic year 2023-2024.

MR.V.SRINIVAS
Coordinator

Dr.E.SUDHARSHAN
Head of the Department

ACKNOWLEDGEMENT

First of all, I'm grateful to The Almighty God for establishing me to complete this Seminar report. I pay respect and love to my parents and all my family members and friends for their love and encouragement throughout my career.

I wish to express my sincere thanks to **MR.V.SRINIVAS sir**, Coordinator Sumathi Reddy Institute of Technology for Women, for providing me with all the necessary facilities.

I place on record my sincere gratitude to **Dr.I.RAJASRI REDDY** mam, Principal, for her constant encouragement.

I deeply express my sincere thanks to our Head of the Department,

DR E SUDHARSHAN Sir for encouraging and allowing me to present the Seminar on the topic A Malware Detection Approach Using Autoencoder in Deep Learning at the department premises for the partial fulfillment of the requirements leading to the award of the B.Tech degree.

It is my privilege to express sincere regards to our project guide **PROF.J.VEDIKA** mam, for the valuable input, able guidance, encouragement, whole-hearted co-operation and constructive criticism throughout the duration of seminar.

I take this opportunity to thank all our faculty, who have directly or indirectly helped me. Finally, I express my thanks to our friends for their co-operation and support.

Sincerely,
Dasi Susmitha
216Y5A6601
CSM
SRITW

ABSTRACT

Today, in the field of malware detection, the expanding limitations of traditional detection methods and the increasing accuracy of detection methods designed on the basis of artificial intelligence algorithms are driving research findings in this area in favor of the latter. Therefore, we propose a novel malware detection model in this paper. This model combines a grey-scale image representation of malware with an autoencoder network in a deep learning model, analyses the feasibility of the grey-scale image approach of malware based on the reconstruction error of the autoencoder, and uses the dimensionality reduction features of the autoencoder to achieve the classification of malware from benign software. The proposed detection model achieved an accuracy of 96% and a stable F-score of about 96% by using the Android-side dataset we collected, which outperformed some traditional machine learning detection algorithms.

TABLEOFCONTENTS

1. INTRODUCTION

The rapid growth of the software industry, driven by mobile internet technology, has given rise to an alarming increase in malware threats. As of 2019, there were over 13.5 million cases of mobile internet malware, with nearly 2.8 million new cases emerging that year, as reported in the China Internet Annual Network Security Report. Android, with its open application market, has become a prime target for mobile-based malware attacks, necessitating the development of an efficient and innovative mobile malware detection approach to combat this growing problem. Traditional malware detection techniques, reliant on manually configured detection rules, struggle to keep pace with the ever-evolving landscape of malware variants. In recent years, the integration of artificial intelligence (AI) algorithms with malware detection has exhibited remarkable improvements in accuracy, robustness, and adaptability, surpassing conventional methods. This research delves into malware detection systems grounded in AI algorithms, focusing on two primary phases: data pre-processing and model classification.

The data pre-processing phase encompasses static and dynamic feature extraction methods. Static extraction involves analyzing features without executing the software program, such as bytecode, file headers, API calls, application interfaces, and permissions. Dynamic extraction, on the other hand, observes the behavioral activities of software during runtime, providing more accurate insights. While dynamic methods yield superior accuracy, they are often challenged by the complexity of virtual environments used by some malware, impacting the stability and efficiency of classification models. In the model training and classification phase, two prominent approaches emerge: machine learning algorithms and deep learning models. Machine learning-based methods utilize common algorithms like Support Vector Machines, K-Nearest Neighbors, Naive Bayes, and decision trees, often incorporating feature learning to achieve high detection accuracy. Deep learning models, including convolutional neural networks (CNNs) and recurrent neural networks (RNNs), have gained prominence in malware detection. CNNs, known for their ability to extract local features, exhibit high performance and scalability. RNNs, while accurate, can be susceptible to adversarial attacks.

2. LITERATURESURVEY

Malware detection is a critical aspect of cybersecurity, and deep learning techniques have shown promise in improving the accuracy and efficiency of this task. One notable approach involves the use of autoencoders, a type of neural network architecture. Autoencoders are neural networks designed for unsupervised learning tasks, where they aim to reconstruct their input data. In the context of malware detection, researchers have found autoencoders to be effective for feature extraction and anomaly detection. By training an autoencoder on a large dataset of benign files, it learns to capture the underlying patterns and structure of legitimate software. Consequently, when presented with potentially malicious code, the autoencoder can identify anomalies in the data that deviate from the learned patterns.

Autoencoders can be implemented as Variational Autoencoders (VAEs) or convolutional autoencoders, depending on the nature of the data (binary or image-based). VAEs offer probabilistic representations, which can be advantageous in quantifying uncertainty in the predictions. Additionally, combining autoencoders with other deep learning techniques, such as recurrent neural networks (RNNs) or Long Short-Term Memory (LSTM) networks, further improves the detection accuracy by considering temporal aspects of malware behavior. One notable advantage of using autoencoders in malware detection is their ability to adapt to new threats. Since they focus on learning data patterns rather than relying on predefined signatures, they can potentially detect zero-day attacks or previously unseen malware. Researchers have also explored the use of adversarial training to enhance the robustness of autoencoder-based detectors against evasion attempts by attackers.

Overall, autoencoders in deep learning offer a promising approach to malware detection, especially when combined with other techniques and methodologies. Their adaptability and ability to detect both known and unknown threats make them a valuable tool in the ongoing battle against cyber threats. However, the effectiveness of these models depends on the quality and diversity of the training data and the careful design of the neural network

architecture, which remains a topic of ongoing research in the field of cybersecurity.

There are 2 main phases about malware detection techniques using artificial intelligence algorithms: the data preprocessing phase, which focuses on the extraction of software features, and the model classification phase, which uses the feature data to train the model to complete the classification task. In the data pre-processing phase, the common extraction methods include static extraction and dynamic extraction about feature data. Static extraction of features means extracting features without running the software programming ways that include extracting bytecode, file header information, API call information application interface semantic information contained in it. The detection method using such features is included less overhead and stable, such as MaMadroid proposed by Maricontiet al, and a malware detection method proposed by Wenjin Li et al. which used Android-side application permission information, API call information and other static data for malware detection.

Compared with static extraction, the dynamic approach analyzes the behavioral activities of software runtime. Therefore, the extracted features are more accurate, such as the DL-Droid proposed by Mohammed et al, who used software log files running on real devices to extract feature data. They used more than 30,000 applications to extract feature data, and the accuracy was as high as 99.6%. Tobiyama et al used recurrent neural networks to extract feature from the temporal data of the processes when the malware program was running, and then used convolutional neural networks to classify them with a high accuracy of 96%. However, many malware program hide their malicious behavior in a virtual environment, and the dynamic virtual operating environment required to make them behave maliciously is more demanding and complex, so the classification model using such features is less stable in accuracy and more overheads information, application permission information, etc.

The main principle of static analysis features is to obtain the source code or bytecode of the program through software decompilation and analyzes the semantic features an in the model training and classification phase, the main approaches include malware detection methods based on machine learning algorithms and deep learning models. The methods based on machine learning algorithms mainly use common machine learning algorithms as classification models, like Wang et al, used five machine learning models for software classification, namely Support Vector Machine (SVM), K-Nearest

Neighbour (KNN), Naive Bayes (NB), Classification Regression Tree (CART) and Random Forest (RF). Kumar et al proposed a feature learning model using various machine learning algorithms to achieve detection of malware with low overhead and high accuracy. RepassDroid extracted various APIs with sensitive trigger points and basic software permissions as datasets to train a machine learning model for detection. They used 24,288samples for training and testing, and the experimental results show that their method had satisfactory results with accuracy rates of 97.7% and 93.3%, respectively. Yerima et al. Used a Bayesian classifier as classification model, andLi et al, used a decision tree to construct a model to achieve classification and detection of malware.

Malware detection methods based on deep learning models mainly use neural networks, recurrent neural networks and convolutional neural networks to implement malware detection. The malware detection methods applying recurrent neural network models are the most common. The methods based on this network structure usually encode all API instructions of the malware as one-hot vectors and put them into the model as input data. Long-term short-term memory (LSTM) networks have shown on the metric of high accuracy. However, RNNs are vulnerable to attacks. An attacker mimics the RNN used in MDS based on inputs and outputs and adds redundant API calls using the adversarial RNN in an adversarial attack. Files injected using redundant API calls can easily by pass RNN detection. Despite the high accuracy of RNNs, the reliability of the results generated by RNNs may still be questionable in malware detection.

Convolutional neural network scan extract location non-specific local features from fixed size, high-dimensional tensor-type data. As a result, it has been used and shown excellent performance in computer vision research, and there are also many research applications in the field of malware detection. Mahoud et al used2D-CNN and 3D-CNN as classification models and used various detection data extracted from dynamic environments as feature data, with an accuracy of up to 90%. Xiao et al, used CNN to understand the characteristics of Android malware from Dalvik bytecode.

3. WORKING

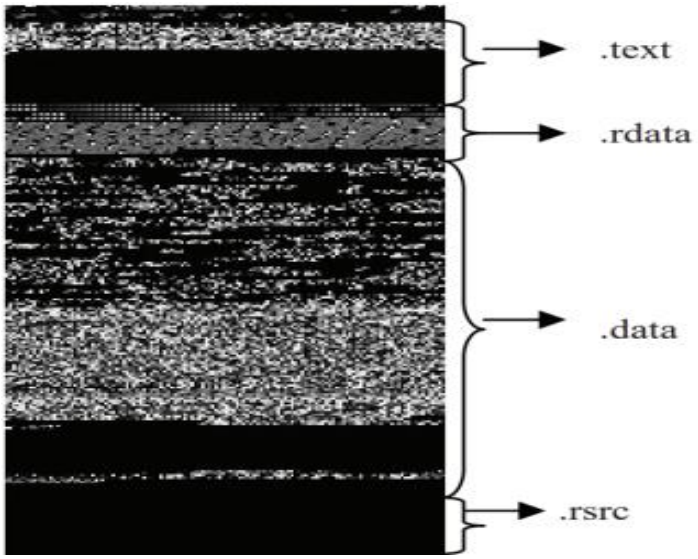
The work related to the generation of malware images and the static malware detection method based on deep learning models. Therefore, this section contains 2 parts, the first part is the malware image generation scheme and the second part is the static malware detection scheme based on deep learning models. In the feature extraction phase for malware detection, neural networks can also be used to extract the corresponding features of the software in addition to extracting the corresponding static feature information, such as API calls, permission information, etc, and dynamic feature information, such as network activity, log files, etc. This feature extraction solution is more automated and simpler than other manual feature extraction methods. Automatic extraction of software features using neural networks requires consideration of the data representation. So that it can better extract the key features and ensure the accuracy of the test results.

A feasible solution is the use of images, where the program is transformed into an image and handed over to the neural net to extract the features. The similarity of software structures is reflected by the similarity of textures between corresponding images, such as the malware picture representation scheme proposed by Natarij et al, they transformed the binary code of the malware into the form of a 2-dimensional matrix, and it can be represented in the form of a grey-scale graph since the numerical range of its transformed matrix is [0, 255], where data of different structures have different textures. Yan et al generated grayscale images from malware files while decompiling to obtain the software opcode sequences, trained the grayscale images using convolutional neural networks, learned the opcode sequences using long and short-term memory networks, and conducted experiments on more than 40,000 samples with an accuracy of 99.88%.

They used bilinear interpolation to resize the images to ensure that the size of the grayscale images input to the training network should be the same size. Proposed a malware detection method based on image recognition. They converted malware into RGB images and classified them using CNN and spatial pyramid pooling (SPP) layer. Experimental evaluation showed that the malware detection method designed based on

RGB images is highly accurate and resistant to redundant API injection attacks. ASLAN focused on the design of the network architecture of the detection model by converting files of software samples into grey-scale maps of malware, training and detecting them using a hybrid network structure, and testing them on the Malign dataset with an accuracy of 97.98%. Nisa et al, used distinctive

Pre-trained models (Alex Net and Inception-V3) for feature extraction, a hybrid model consisting of deep learning algorithms and traditional machine learning algorithms to achieve 99.3% accuracy for a 25-class malware classification task using data augmentation based on affine image transformation. Singh et al. used 15 different combinations of Android malware image to identify and classify Android malware, and machine learning algorithms were used to analyze grey-scale malware images instead of the Softmax layer of CNN such as K-NearestNeighbour (KNN), Support Vector Machine (SVM) and Random Forest (RF). The classification results showed that the method achieved a correct classification rate of 92.59%.



These works focus on how to convert software of different sizes into images of the same size, and need to consider the challenge of how to do the best possible job of reducing redundancy in the process of generating the images. The difference between our work and previous work is that we try to extract the binary code of the method field in the software and convert some of the information into byte code to complete the generation of the grey-scale image. Analyzing the feasibility of such a scheme is a major part of our research.

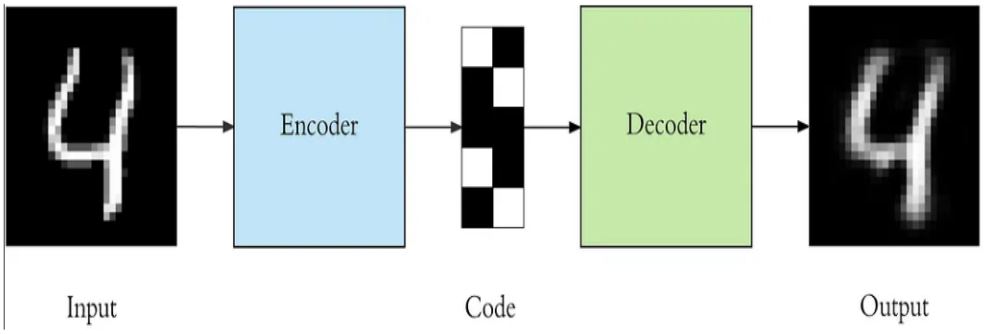
Deep learning models can show better performance on classification and prediction tasks, so they have been widely used in many research areas, such as recommendation systems, privacy protection, image recognition, and natural language processing. In the field of malware detection, deep learning models also have a wide range of applications. The proposed malware detection scheme is related to the static feature extraction of software samples and the use of deep learning networks as classification detection models. Therefore, we present some noteworthy work in the area of malware detection models based on deep learning models. There are two reasons for choosing the static analysis approach to extract software file features.

Firstly, static analysis is intuitive and comprehensive, as compared to dynamic extraction efforts, static analysis does not need to consider when malware needs any trigger conditions to exhibit malicious behavior, and its underlying source code intuitively contains the functional features of malware. Secondly, static analysis is faster and more efficient than dynamic detection, which takes a long time to run the malware program in order to record all kinds of data and is inefficient when dealing with a large number of software samples, whereas static analysis can extract features from a large number of software samples in a short period of time, which makes practical sense. The deep learning model was chosen because of its ability to generalize and detect previously unseen malware samples with high accuracy.

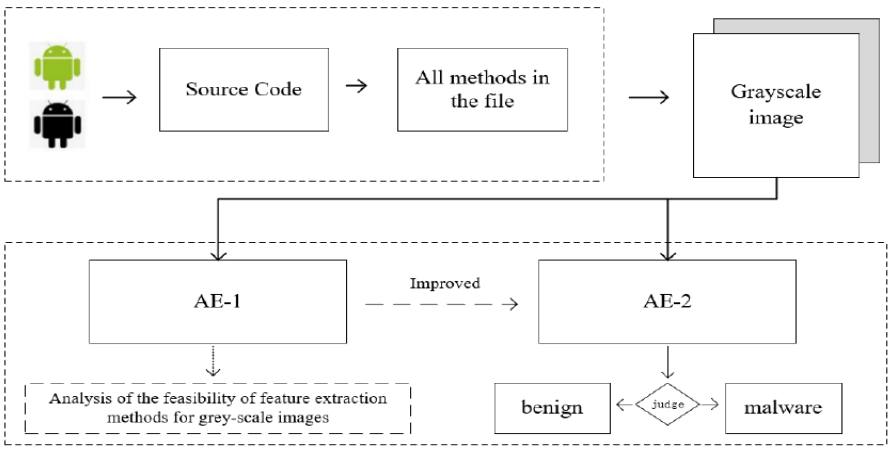
We propose a approach to malware detection, which is designed based on the automatic encoder network. This illustrates the overall structure and main tasks of our malware detection method. First, benign files and malware are transformed into corresponding grayscale images by decompiling the APK files, the binary codes are extracted from methods in software, then converting them into decimal data by bytes, which are filled with pixel value. Afterwards, the greyscaleimages are passed through 2 deep learning networks in order to complete 2 tasks. The first deep learning network named automatic encoder network, which we use to analyze the feasibility of using grey-scale images to represent the corresponding features of software's, and the second deep learning network is automatic encoder network, which we use to perform the task of classifying malicious software from benign software's.

4. STRUCTURE OF AN AUTOENCODER

Autoencoders are a specific type of feed forward neural networks where the input is the same as the output. They compress the input into a lower-dimensional code and then reconstruct the output from this representation. The code is a compact "summary" or "compression" of the input, also called the latent-space representation. An autoencoder consists of 3 components: encoder, code and decoder. The encoder compresses the input and produces the code, the decoder then reconstructs the input only using this code.



Autoencoders are very useful in the field of unsupervised machine learning. You can use them to compress the data and reduce its dimensionality. The main difference between Autoencoders and Principle Component Analysis (PCA) is that while PCA finds the directions along which you can project the data with maximum variance, Autoencoders reconstruct our original input given just a compressed version of it. If anyone needs the original data can reconstruct it from the compressed data using an autoencoder. Autoencoders is an unsupervised neural network that you can use to generate a compressed version of the input data. It is done by taking in an image and trying to predict the same image as output, thus reconstructing the image from its compressed bottleneck region. The primary use for autoencoders like these is generating a latent space or bottleneck, which forms a compressed substitute of the input data and can be easily decompressed back with the help of the network when needed.



However, the conversion of software binary codes into grey-scale images has some drawbacks. Although the software binary code contains a variety of feature, it also contains large amount of works focus on how to convert software of different sizes into images of the same size, and need to consider the sticking points of how to do the best possible job of reducing redundancy in the process of generating the images. The difference between our work and previous work is that we try to extract the binary code of the method in the software and convert some of the information into byte code to complete the generation of the grey-scale image. Analyzing the feasibility of such a scheme is a major part of our research. The redundant information causes high pre-processing overhead and reduces the accuracy and robustness of the model classification at the later stage.

Autoencoders can be stacked and trained in a progressive way, we train an autoencoder and then we take the middle layer generated by the AE and use it as input for another AE and so on. This is the first step towards deep learning, the stacked autoencoders will learn how to represent data, the first level will have a basic representation, and the second level will combine that representation to create a higher-level representation and so on. Think about images, a first-level autoencoder will learn to detect borders as features, the second level will combine those borders to learn traces and patterns, etc. An autoencoder has a lot of freedom and that usually means our AE can overfit the data because it has just too many ways to represent it. To constrain this we should use sparse autoencoders where a non-sparsity penalty is added to the cost function. In general, when we talk about autoencoders we are really talking about sparse autoencoders.

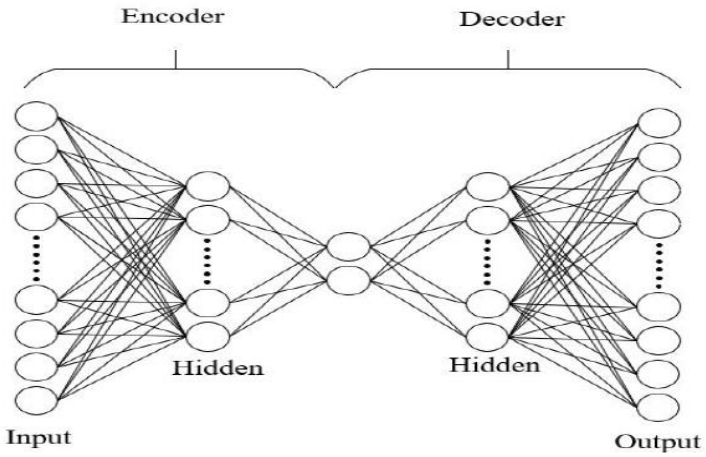
For this reason, we decompile the software and instead of converting the software binary data directly into a greyscaleimage. We extract all the methods in the software and convert the byte code of methods into a grayscale image, in any blank areas with zero. The advantage of this is two fold. Firstly, these methods contain various actions of the software, such as sending network data, reading private information on the phone, writing data to the phone's ROM and harddrive, and can be used to visually represent malicious actions in a greyscale image without setting up a dynamic runtime environment. The second point is that we have reduced the redundancy of using images to represent malware compared to previous grey-scale image processing, making the subsequent classification of the model more accurate and stable.

The autoencoder network structure is a special kind of unsupervised neural network in a deep learning model. It consists of an encoding network and a decoding network, the encoding network achieves the effect of dimensionality reduction and compression, and the decoding network achieves the purpose of reconstructing the input. Its loss function is denied as the error value between the original input and the model output corresponding to the original input, and minimizing its loss function by means of training and gradient updating is the operation process of the autoencoder network. Borghesi et al, used autoencoders to enable anomaly detection in large computer systems. Their results show that the autoencoder can monitor anomalies that were never noticed before based on previous log records with an accuracy of between 88% and 96%.Angelo et al. propose a malware detection system for Android based on an auto encoding network. They put sequences of API calls from the application as input into an autoencoder network to complete feature extraction, then used a neural network to train and classify features. Their system achieves higher accuracy than complex traditional machine learning methods such as J48, Naive Bayesian and MLP.

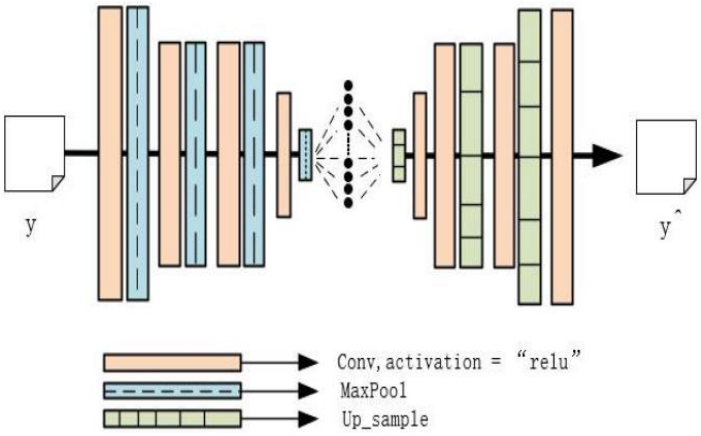
4.1 THE FIRST AUTOMATIC ENCODER STRUCTURE (AE-1):

The structure of model AE-1 is shown in Fig. 4, and consists of convolutional layers, pooling layers. We designed 2 model structures and named them AE-1 andAE-2 respectively, the design sequence is AE-1 and thenAE-2. The main purpose of designing the AE-1 network is to use it to analyze the feasibility of feature extraction methods for

grey-scale images, and the purpose of designing theAE-2 network is to use it for malware detection. The reason for designing the 2 networks is that the AE-1 network exhibited more drawbacks and less stability for the experimental aspects of the classification task, so we improved on the AE-1 network and proposed the AE-2 network. It is worth noting that the AE-1 network is trained in an unsupervised manner and no software samples are labeled, while the AE-2network is trained in a supervised manner and requires labeling of malicious and benign software samples.



The schematic representation of an autoencoder.



The first automatic encoder structure (AE-1)

The structure of model AE-1 is, and consists of convolutional layers, pooling layers and up-sampling layers. The activation function uses the relu function and the MAE loss function with the Equ.(1):

$$l_r = \frac{1}{2n}\sum_i (y_i - y_i^r)^2 \qquad (1)$$

Model AE-1 analyzes whether it can reconstruct the mal- ware feature image based on the magnitude of similarity between the original malware feature image and the reconstructed image that has been reconstructed through the autoencoder network. We determine whether AE-1 is able to perform this task by analyzing the numerical magnitude of its similarity, and this measure of similarity is expressed by defining the Similar Error as the following Equ.(2).

$$SimiliarError = \frac{\sum_{i=0}^{N}(|y_r^i - y_g^i|)}{N} \qquad (2)$$

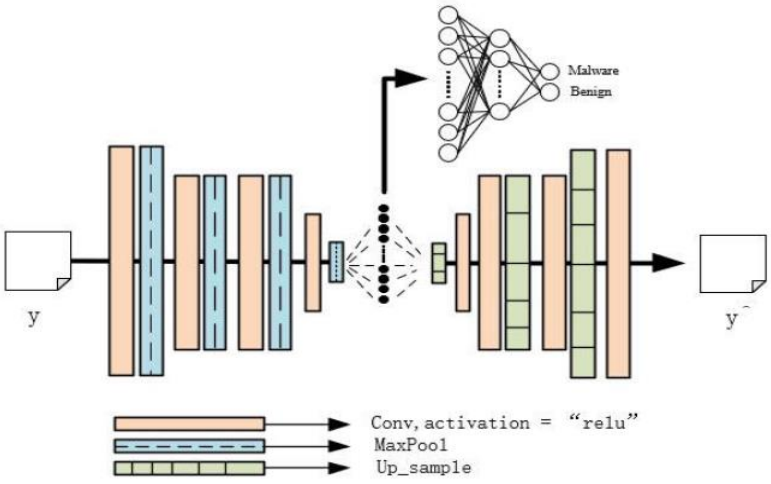
Wherey^i_r is a pixel of the original image, y^i_g is the pixel corresponding to the original position in the image generated by the autoencoder, and N is all the pixel points of an image. The theoretical basis for determining whether AE-1 can perform this task based on the numerical magnitude of the Similar Error is that only unlabeled malware datasets are employed during the training phase of the autoencoder network. In the predictive classification phase, if a feature image corresponds to a category that is malware, then the reconstructed image it generates via the autoencoder network is the same as the original.

On the contrary, if a feature image corresponds to a category belonging to benign software, the similarity between the reconstructed image generated by model and the original image will be low and the Similar Error value will be enlarged, since the structure of a feature image transformed by benign software is very different from the structure of a feature image transformed by malware.

For example, if we ask an expert in real life to focus on malware without studying the characteristics of benign software, he can easily distinguish the difference between benign and malware if he is knowledgeable in the high-dimensional characteristics of malware and then looks at benign software. If the error value of Similar Error generated by the two types of software after such an autoencoder have a huge difference, then we can use this to make sure that the autoencoder network can indeed reconstruct the corresponding feature images of the two types of software better.

4.2 THE SECOND AUTOMATIC ENCODER STRUCTURE (AE-2):

The structure of model AE-2 is in which the autoencoder network structure is similar to model AE-1.The only difference is that we have an external multi-layer perceptron network to facilitate classification and experimental evaluation. We first extract the high-dimensional features corresponding to malware and benign software from model AE-1 by pre-training, then extract the output from the hidden layer of model AE-1 and use it for the training of the multilayer perceptron network.



The second automatic encoder structure (AE-2)

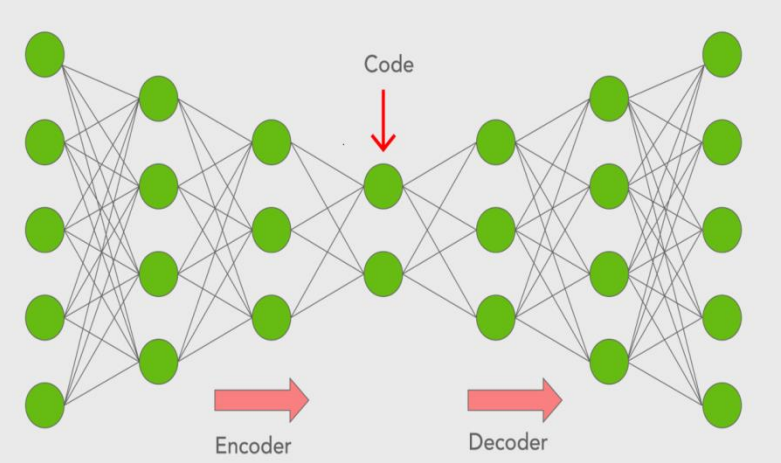
5. ARCHITECURE OF AN AUTOENCODER

Auto encoder is a type of neural network where the output layer has the same dimensionality as the input layer. In simpler words, the number of output units in the output layer is equal to the number of input units in the input layer. An auto encoder replicates the data from the input to the output in an unsupervised manner and is therefore sometimes referred to as a replicator neural network. The auto encoders reconstruct each dimension of the input by passing it through the network. It may seem trivial to use a neural network for the purpose of replicating the input, but during the replication process, the size of the input is reduced into its smaller representation. The middle layers of the neural network have a fewer number of units as compared to that of input or output layers. Therefore, the middle layers hold the reduced representation of the input. The output is reconstructed from this reduced representation of the input.An auto encoder consists of three components:

Encoder: An encoder is a feed forward, fully connected neural network that compresses the input into a latent space representation and encodes the input image as a compressed representation in a reduced dimension. The compressed image is the distorted version of the original image.

Code: This part of the network contains the reduced representation of the input that is fed into the decoder.

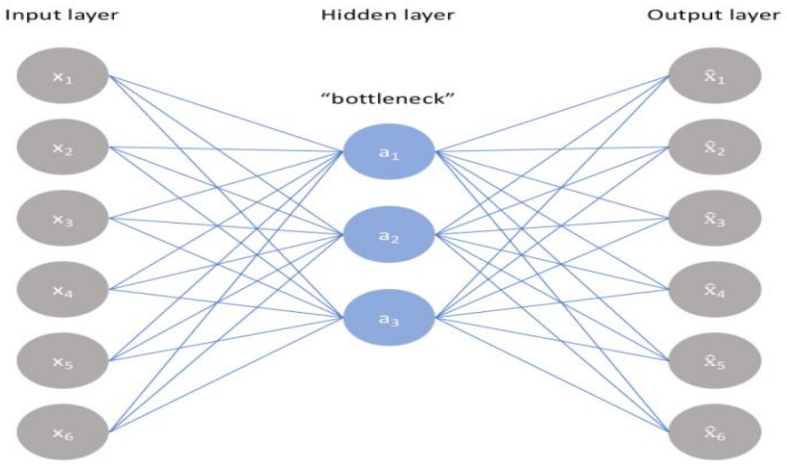
Decoder: Decoder is also a feed forward network like the encoder and has a similar structure to the encoder. This network is responsible for reconstructing the input back to the original dimensions from the code.



First, the input goes through the encoder where it is compressed and stored in the layer called Code, and then the decoder decompresses the original input from the code. The main objective of the autoencoder is to get an output identical to the input. Note that the decoder architecture is the mirror image of the encoder. This is not a requirement but it’s typically the case. The only requirement is the dimensionality of the input and output must be the same.

An autoencoder is a neural network architecture used for unsupervised learning and dimensionality reduction. It consists of two main components, an encoder and a decoder. The encoder compresses input data into a lower-dimensional representation, while the decoder aims to reconstruct the original input from this compressed representation. The bottleneck layer in the encoder captures the most important features. The primary objective is to minimize the difference between the input data and the reconstructed output, typically using a loss function like mean squared error. Autoencoders are versatile and can be adapted for various data types, making them valuable for tasks like data denoising, feature learning, and anomaly detection.

An autoencoder is a type of artificial neural network used for unsupervised learning and dimensionality reduction. It consists of two main parts: an encoder and a decoder. The encoder compresses the input data into a lower-dimensional representation, and the decoder attempts to reconstruct the original input from this compressed representation. The bottleneck layer, within the encoder, holds the most critical features. The network's primary objective is to minimize the difference between the input data and the reconstructed output, usually employing a loss function like mean squared error. Autoencoders can be adapted for various data types, such as images, text, or time series, by adjusting their architectures accordingly.



The encoder in an autoencoder is responsible for compressing the input data into a lower-dimensional representation. It accomplishes this by applying a series of layers that gradually reduce the dimensionality while extracting essential features from the input. These features are then stored in the bottleneck layer, also known as the latent space. The bottleneck layer is a crucial component of the autoencoder where the most critical information is retained in a compact form. On the other side, the decoder takes the information from the bottleneck layer and attempts to reconstruct the original input data. It mirrors the architecture of the encoder but in reverse, expanding the compressed representation back to the original dimension. The objective of this process is to minimize the difference between the input data and the reconstructed output. Autoencoders are valuable for various applications, such as data denoising, feature extraction, and anomaly detection, due to their ability to capture meaningful representations within the bottleneck layer.

The encoder serves as the initial stage of an autoencoder, responsible for reducing the dimensionality of the input data and extracting meaningful features. It comprises one or more layers, often utilizing activation functions like ReLU (Rectified Linear Unit), which enable it to learn complex patterns in the data. As the data progresses through the encoder's layers, it undergoes a gradual transformation, capturing increasingly abstract representations of the input. The final layer of the encoder is the bottleneck layer, also known as the latent space or coding layer. This layer holds a compact representation of the input data, where the essential features are encoded into a lower-dimensional space.

The bottleneck layer is a critical component of the autoencoder architecture. It acts as a bottleneck, forcing the network to learn a compressed and informative representation of the data. This is achieved by limiting the number of neurons or units in this layer, thus compelling the network to capture only the most salient information. The quality of the information stored in the bottleneck layer largely determines the autoencoder's performance. If it succeeds in retaining vital information while discarding noise or irrelevant details, the reconstruction process by the decoder will be more accurate.

The decoder, the counterpart to the encoder, takes the compressed representation from the bottleneck layer and attempts to reconstruct the original input data. This part of the autoencoder typically mirrors the architecture of the encoder but in reverse. It involves layers that gradually expand the dimensionality of the data, refining it to resemble the original input. The reconstruction is achieved through the application of activation functions and specific network parameters. The primary objective during training is to minimize the difference between the input data and the output generated by the decoder.

This process encourages the autoencoder to capture the most informative features in the bottleneck layer to facilitate accurate reconstruction. Autoencoders, with their encoder, bottleneck layer, and decoder, are versatile tools used in a wide range of applications. They can uncover hidden patterns in data, reduce data dimensionality for efficiency, and even denoise corrupted information. The quality of the bottleneck representation is a critical factor in determining the success of the autoencoder for various tasks, making it a key area of focus in their design and training.

The encoder is a set of convolutional blocks followed by pooling modules that compress the input to the model into a compact section called the bottleneck. The bottleneck is followed by the decoder that consists of a series of up sampling modules to bring the compressed feature back into the form of an image. In case of simple autoencoders, the output is expected to be the same as the input data with reduced noise. However, for variational autoencoders it is a completely new image, formed with information the model has been provided as input.

6. PERFORMANCE EVALUATION

We evaluate the proposed approach through experiments under different indicators. This section includes four parts, namely experimental setup, data feature extraction, effectiveness analysis of reconstructing malware images and performance analysis of the detection model.

A) EXPERIMENTALSETUP:

This subsection focuses on information related to the experimental setup and, for this purpose, is divided into 3 sections,

TABLE 1. Experimental environment setup

Experimental environment setup	Requirements
CPU	Intel CoreTM i5-8300
RAM	16GB RAM
GPU	GeForce GTX 1060 MQ
Operating System	64-bit windows10 operating system
Programming Frameworks	Tensorflow 2.1, and Python 3.7

1) EXPERIMENTAL ENVIRONMENT SETUP :

The details are shown in table 1 for information on the experimental environment. Our experiments were conducted using an Intel CoreTM i5-8300 machine with 16GB RAM, and GeForce GTX 1060 MQ. The machine had a 64-bit windows10operating system. We used Keras, Tensor flow 2.1, and Python 3.7 for programming purposes.

2) DATASET:

To evaluate the performance of the proposed model, we collected benign software from the Google App Store and malware from VirusShare, where benign software consisted of 10 categories such as office, video, gaming, nance, photography and

reading, and malware included datasets for APK categories released in 2016, 2017 and 2018.Virus Total scans a random sample of software to determine that they are correctly labled.

We divided them into 3 types of datasets according to their purpose, namely: (1) Dataset-1, this dataset is used for training and evaluation of AE-1 models, which includes8121 malware and 2000 benign software's. (2) Dataset-2, this dataset is used for training, validation and testing of theAE-2 model and contains 8121 malware and 7015 benign software. (3) Dataset-3, this dataset is used to analyze the detection performance of the AE-2 model on unseen software and includes 5,384 malware and 5,000 benign software. It is worth noting that when we divided Dataset-2 andDataset-3, we deliberately put older software samples intoDataset-2 for training, e.g. malware from 2016, and newer releases into Dataset-3, e.g. 2017, 2018. The purpose of this isto simulates the scenario when the model detects new software samples released in the future and to facilitate the analysis of its performance.

3) TRAIN AND TEST DETAILS :

The AE-1 network is used for the task of analyzing the performance of the autoencoder to reconstruct feature images, and detailed parameters of model AE-1 are shown in Table 2. The Adam optimization algorithm is used in the training phase and we set the learning learning rate to be 1e-4, the epoch is 100. We divide the dataset-1 into 3 parts, the training dataset DTrain, which contains a partial dataset of the collected.

TABLE 2. Parameters of AE-1 model:

Layer	Output shape	Parameters
input	[(None,500,500,1)]	0
maxpooling2d	[(None,250,250,1)]	0
conv_2d	[(None,250,250,8)]	80
maxpooling2d	[(None,125,125,8)]	0
conv_2d	[(None,125,125,8)]	584
maxpooling2d	[(None,63,63,8)]	0
conv_2d	[(None,63,63,6)]	584
up_sampling2d	[(None,126,126,8)]	0
conv_2d	[(None,126,126,8)]	584
up_sampling2d	[(None,252,252,8)]	0
$conv_2d$	[(None,250,250,16)]	1168
up_sampling2d	[(None,500,500,16)]	0
conv_2d	[(None,500,500,1)]	145

Malware software, the malware test set DTest_mal , which contains partial dataset of the collected malware, and the benign software test set DTest_benign, which contains a dataset of the collected benign software files.

The AE-1 network use training dataset DTrain for the training task, then use the malware test set DTest_mal and the benign software test set DTest_benign for the test task. If the new input of test set is similar to the input of the dataset used in the training phase, then the reconstruction error for this test set is very small. Conversely, if the new inputs of test set are different from the inputs to the dataset used in the training phase, then this test set will exhibit a very large reconstruction error. The large difference in the error data produced by these2 test sets after AE-1 is exactly what we are experimenting with. Since our hypothesis is based on the theory that malware is all similar and benign software is not similar to malware, in practice, the different functional characteristics exhibited between malware families in the malware dataset and the large redundancy characteristics contained in the software dataset can lead to experimental results exhibiting large instabilities. For this reason, we are more interested in the relative differences between the 2 test sets than in the absolute errors they exhibit.

The AE-2 network is used for the task of analyzing the performance of the detection model. For the overall dataset partitioning, we used 80% of the Dataset-2 as the training set and 20% as the test set. In the training phase, the training set was trained and validated using k-fold cross-validation, with k D 6, meaning that 5/6 of the training set was used for training and 1/6 for validation, repeated 6 times, and finally the average was taken. In the testing phase, the test set is used for testing. Minutes are used as units for training time. The Adam optimization algorithm, learning rate of 0.0001 and epoch of 100 are chosed in AE-2's training. The variety of evaluation metrics such as FPR, TPR, ACC, Precision and F-score are used in the model evaluation test phase, which are calculated as shown below,

$$FPR = FP/(FP + TP) \quad (3)$$

$$TPR = Recall = TP/(TP + FN) \quad (4)$$

$$Acc = (TP + TN)/(TP + TN + FP + FN) \quad (5)$$

$$Precision = TP/(TP + FP) \quad (6)$$

$$F1 - score = 2 \cdot Precision \cdot Recall/(Precision + Recall) \quad (7)$$

B) DATA FEATURE EXTRACTION:

We used the Androguard tool to complete the datapre-processing task of the model, extracting the source code of all the class files in the APK file through the Androguard analysis framework, extracting the byte codes of all the methods and converting them into the decimal data needed for the corresponding grey-scale images. In the sample dataset of software collected, we chose files with as small a data size as possible to ensure that we could standardize the size of all images. Based on this approach, all software is converted into the feature images we need during the data pre-processing phase.

C) EFFECTIVENESS ANALYSIS OF RECONSTRUCTING MALWARE IMAGES:

We evaluate the effectiveness of reconstructing malware images by analyzing the overall error distribution in malware and benign reconstruct malware images. The below figure shows the overall error distribution for the 2 test sets. The reconstructed error value generated by each software after the encoder network is normalized and expressed as value on the Y-axis. We normalize by adding up the error value for each pixel point corresponding to the feature image of the malware and dividing by the total. In the line statistics graph, the blue line represents the error trend for the overall DTest_mal and the yellow line represents the error trend for the overall DTest_benign. The error is not exactly zero due to the inherent variability contained in the dataset and the redundancy of the software files.

However, as can be seen in Fig. 6, the overall error trend is stable for the malware dataset represented by the blue line, whereas the overall error trend for the benign software test set represented by the yellow line is unstable and actuates widely, and the relative difference between the mean value of the errors presented by the two datasets is large. Thus, our theory is plausible. We conducted a quantitative analysis of the two test sets and normalized the value and presented them in Table 3,where the normalization was done by calculating the mean absolute error (MAE) and root mean square error (RMSE) of the test set after making the error of the training set equal to 1.

The experimental data, as described in Table 3, showed that the normalized MAE and normalized RMSE produced by the malware test set were close to 1, indicating that the DTest_malware similar to the DTrain, while the benign test set produced a normalized MAE and normalized RMSE greater than 1 and also greater than the value of the training set, indicating that the benign software test set was not similar to the training set. The normalized MAE and normalized RMSE for the software test set were greater than 1 and also greater than the value for the malware test set, indicating that the DTest_benign was,

TABLE 3. Quantitative analysis of two datasets.

datasets	D_{Test_mal}	D_{Test_benign}
Normalized MAE	1.12	2.02
Normalized	1.25	2.89

not similar to the DTrain, and this quantitative analysis also demonstrated that the network structure would show similar results on the invisible data set. Based on this experiment, we can then show that the task of reconstruction can be performed well by the automatic encoder through the pre-processed malware data from our data, and that the automatic encoder can identify high-dimensional features of both benign and malicious software. Then, we implement the subsequent task of classifying malware and benign software.

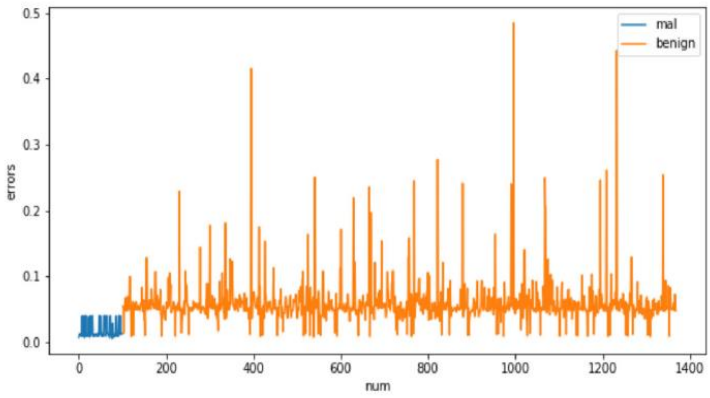


Fig: Reconstruction error for two datasets

D) PERFORMANCE ANALYSIS OF THE DETECTION MODEL:

We use the AE-2 model to analyze the classification performance of autoencoders in this subsection. To the end, we experimented with some similar previous research work for comparative analysis, include detection models using traditional machine learning algorithms, detection models designed based on recurrent neural network, detection models designed based on autoencoder networks and convolutional neural networks and detection models using Malware Images. Among them, the detection model CNN-SPP designed with malware images and convolutional neural networks proposed by He and Kim showed high accuracy on the dataset provided by Seoul National University. The DAE-CNN model proposed by Wang et al, uses an autoencoder network as the data preprocessing model and the output data of the model is used for training and detection of convolutional neural networks, which is similar to our theoretical model, and they show better results on 10,000 benign APPs and 13,000 malicious APPs. We use these 2 types of models as baseline models for comparison experiment with AE-2 on different datasets.

PERFORMANCE COMPARISON OF DIFFERENT MODELS :

The ROC curves in Fig. 7 show the effect of the model on the training set, from which it can be seen that the model exhibits a more stable performance on the training set. The ROC curves in Fig. 8 show the model's performance on the test set VOLUME 10,

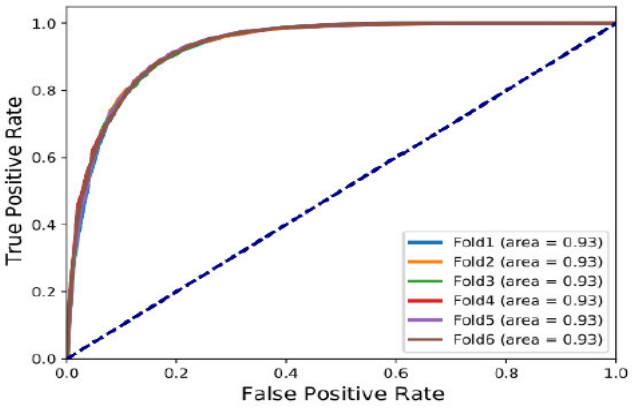


Fig : The ROC curve of AE-2 on training set

which in Dataset-2. We can see that our model out performs the other two. To further examine the detection performance of our model on unseen malware, the Datasets-3 is used as test set for AE-2. The ROC curves are shown in Fig. 9, from which it can be seen that our model shows good accuracy and some feasibility in detecting unseen malware, but also shows some as the software changes with year iteration.

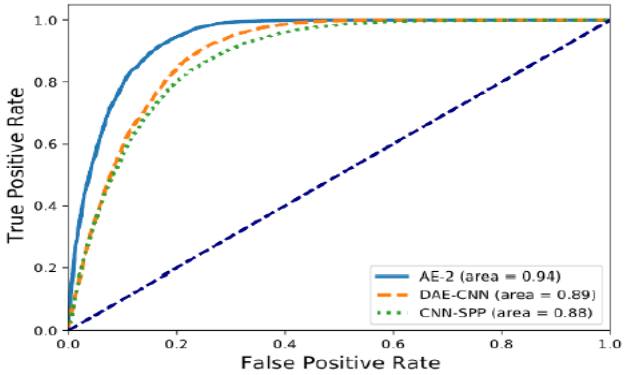


Fig : The ROC curve of different models on the test set

In the experimental results presented in Table 4, the highF1-score value for the AE-2 model demonstrate the feasibility and stability of our solution. The traditional machine learning algorithms and autoencoder have higher F-scores due to the use of multiple ne-grained feature extraction methods, and the efficient feature extraction method is the key to determine the performance of the model. The lower F1-score shown in recurrent neural network model is due to the fact that its feature extraction method is complex and the model built using RNN shows unstable performance on the dataset. The lowerF1-score in convolutional neural network model demonstrated that this way of constructing feature images based on file binary codes contains more redundant information and less distinct features.

2) DETAILED COMPARISON OF MULTIPLE INDICATORS :

After that, we use a variety of common machine learning models and deep learning models to conduct experiments comparing various evaluation metrics. The common machine learning algorithms include support vector machines, decision trees and naive bayes, and the deep learning model named CNN-0, which model consists of one layer of convolution, one layer of pooling and one layer of fully,

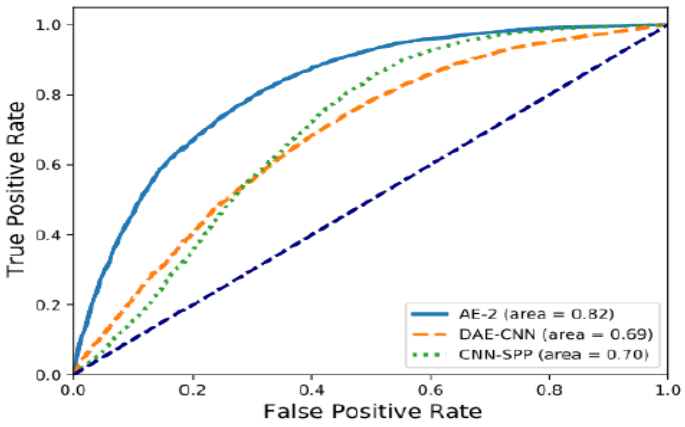


Fig : The ROC curve of different models on the unseen software

Connected neural network, we use it as our benchmark model. The value of accuracy, precision, recall and F-score on the five models. We find that the decision tree outperforms the support vector machine model and the naive Bayes model in terms of accuracy and performance for traditional machine learning detection algorithms through cross-sectional comparisons, while the deep learning model out performs the traditional machine learning algorithm in terms of overall performance. Our model achieves better results in terms of search accuracy and completeness.

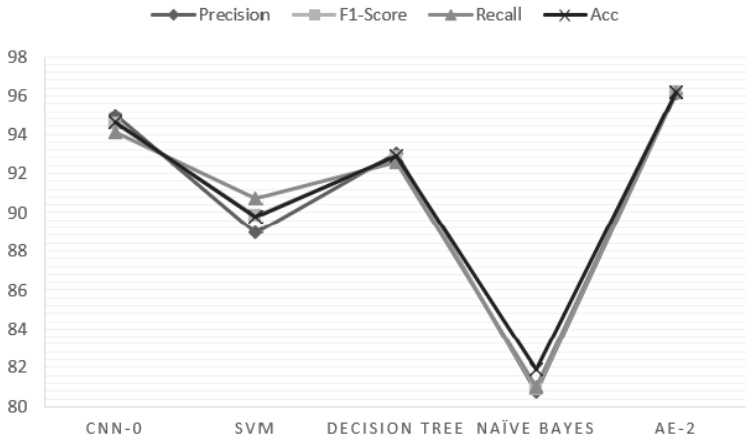


Fig : Comparison results in five different models

This illustrates the performance comparison between the two different deep learning models. It can be seen from the figure that AE-2 spends less training time compared to theCNN-0 model, and the ACC, recall, Precision, and F-score metrics are all very

similar. For FPR, AE-2 shows lower value. Table 5 shows all the experimental data. The training time for our model is 1407.32 mins, which is about 23.45 hours. For the CNN-0 model, the number of parameters in the image data is huge and it takes a significant amount of time, 28.14 hours, after performing the convolution and pooling operations since the structure contains only one layer of convolution and one layer of pooling.

The first point is that the data pre-processing method needs to be improved. Although we did our best to reduce the redundant information carried by the software feature representation, there is still the problem of inefficiency, and the dataset we used is smaller due to the limitations of our experimental environment. The feasibility and effectiveness of this approach on other,

Study,Year	Alogrithm / Model	Features	F1-Score
[22],2018	traditional machine learning algorithms	11 types of static features from each app	92.78
[29],2015	recurrent neural network (RNN)	malware event stream	83.56
[39],2019	convolutional neural networks (CNN)	converts binary malwares files to grayscale/RGB images	86.27
[35],2019	autoencoders	API call sequence	94.69
AE-2	autoencoders and fully connected neural network	the byte code of software methods	96.17

Fig : Performance comparison of different model

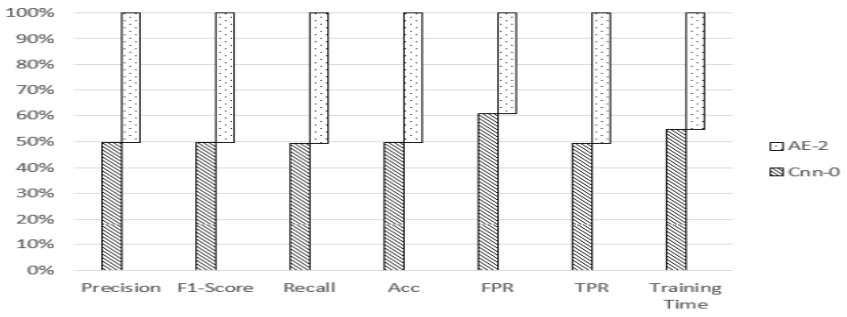


Fig : Performance comparison of 2 deep learning models

The second point is that the instability of detection performance caused by deep learning models is difficult to estimate. Although deep learning algorithms, such as convolutional neural networks, have a promising future in areas such as image recognition and text generation, the use of malware feature data for classification tasks in deep learning models can lead to unstable detection performance, will be reduced.

7. FUTURE SCOPE

The future scope of "A Malware Detection Approach Using Autoencoder in Deep Learning" holds great potential in the field of cybersecurity. Researchers can explore advanced autoencoder architectures, multi-modal detection incorporating various data sources, defenses against adversarial attacks, real-time detection capabilities, strategies for handling imbalanced data, interpretable models, transfer learning, adaptation for IoT and embedded systems, collaboration with the industry, and addressing ethical and regulatory considerations.

These directions promise enhanced capabilities for malware detection, adaptability to diverse scenarios, and ethical, compliant use of this technology. As cyber threats continue to evolve, this research area remains vital in safeguarding digital systems and networks. Researchers can explore advanced model architectures, including deep and recurrent autoencoders, to improve the detection capabilities of these systems. The integration of hybrid models, combining autoencoders with other machine learning techniques, offers an avenue for even greater accuracy.

As the volume of malware data continues to grow, scalable solutions like big data and cloud-based implementations become essential. The development of interpretable models through Explainable AI (XAI) can enhance the transparency of decision-making in malware detection. Dynamic analysis, online learning, and IoT security are other promising areas for research, adapting the technology to emerging cybersecurity challenges.

Additionally, resilient models against evasion techniques and the involvement of human expertise can further enhance system performance. Cross-domain adaptation, regulatory compliance, and adherence to evolving legal frameworks are also critical considerations for the future. In a landscape where cyber threats constantly evolve, this research remains pivotal in safeguarding digital systems and networks from malicious attacks.

8. APPLICATIONS

- Phishing Detection
- Malicious Code Detection
- Android App Security
- Web Application Security
- Malware Detection
- Network Security
- Anomaly Detection
- Improving Antivirus Software
- Data Privacy

9. ADVANTAGES

- Anomaly Detection
- Zero-Day Threat Detection
- Feature Learning
- Flexibility
- Unsupervised Learning
- Scalability
- Machine Learning Integration
- Cost-Effective

10. DISADVANTAGES

- Resource-Intensive
- Lack of Interpretability
- Overfitting
- Security Evasion
- Privacy Concerns
- Lack of Context
- Adversarial Attacks
- Model Interpretability
- Complexity of Hyperparameter Tuning
- Resource-Intensive Deployment

11. CONCLUSION

In this paper, we propose a novel approach to malware detection, which is based on the principle of using grey-scaleimages to represent the features of malware and using an auto-encoder network to design a classification model to achieve malware detection. Experimental results show the feasibility of our proposed approach of converting the bytecode of all methods in software into a greyscale image to represent the features in a software sample. Compared to malware detection methods designed based on traditional machine learning algorithms, our method is more accurate. Our method requires less training time and detection time compared to other malware detection systems designed based on deep learning models. In future work, we will continue to explore more effective methods for representing malware feature images and focus our research on the data pre-processing stage to explore newer malware detection methods.

12.REFERENCES

[1]O. Aslan and A. A. Yilmaz, A new malware classication framework based on deep learning algorithms," IEEE Access, vol. 9, pp. 8793, 87951, 2021.

[2]J. Singh, D. Thakur, F. Ali, T. Gera, and K. S. Kwak,"Deep feature extraction and classication of Android malware images," Sensors, vol. 20, no. 24, p. 7013, Dec. 2020, doi: 10.3390/s20247013.

[3]China Internet Security Research Report. (Nov. 15, 2020).[Online]. Available: https://www.cert.org.cn/publish/main/upload/File/2019Annual%20report.pdf

[4]Y. Ye, T. Li, D. Adjeroh, and S. S. Iyengar, A survey on malwaredetection using data mining techniques," ACM Comput. Surv, vol. 50,no. 3, pp. 1-40, May 2018.

[5]S. Rastogi, K. Bhushan, and B. B. Gupta, Android applications repackagingdetection techniques for smartphone devices," Proc. Comput. Sci.,vol. 78, pp. 26-32, Jan. 2016.

[6](Jan. 15, 2020). The Keras Blog. [Online]. Available: https://blog.keras.io/building-autoencoders-in-keras.html

[7]https://ieeexplore.ieee.org/document/9723074

[8]https://www.mygreatlearning.com/blog/autoencoder/

[9]https://www.imperva.com/learn/application-security/malware-detection-and-removal/

[10]https://www.sentinelone.com/cybersecurity-101/what-is-malware-detection/

[11]https://www.researchgate.net/publication/358972838_A_Malware_Detection_Approach_Using_Autoencoder_in_Deep_Learning

[12]https://www.scribd.com/document/661973213/A-Malware-Detection-Approach-Using-Autoencoder-in-Deep-Learning

[13]https://www.geeksforgeeks.org/malware-and-its-types/